EE 457 Homework 2

Berk Aydemir 2021205033
Şeyma Ünal 2021205027

April 22, 2025

0.1 Powell Function and Set Up

The Powell function, defined as:

$$f(x) = (x_1 + 10x_2)^2 + 5(x_3 - x_4)^2 + (x_2 - 2x_3)^4 + 10(x_1 - x_4)^4 \tag{1}$$

The initial point $x^{(0)} = [3, -1, 0, 1]^T$ and the stop criteria is when gradient norm reaches $\|\nabla f(x)\| \leq 10^{-2}$. We use these conditions for all algorithms.

The algorithms that we will examine:

1. P1. Gradient descent with fixed step sizes
2. P2. Steepest descent with golden section line search
3. P3. Gradient descent with Armijo-Goldstein line search
4. P4. Newton's method
5. P5. Modified Newton's method with golden section line search
6. P6. Modified Newton's method with Armijo-Goldstein line search

1 Algorithms

1.1 Problem 1: Gradient Descent with Fixed Step Size

Gradient descent is a first-order iterative optimization algorithm that minimizes a function by moving in the direction of the negative gradient. The update rule is:

$$x^{(k+1)} = x^{(k)} - \alpha \nabla d \tag{2}$$

where direction d equals to -$\nabla f(x^{(k)})$ where $\alpha > 0$ is a fixed step size. Applying this argument is simple, but it can be slow to converge.

Two fixed step sizes were tested:

- $\alpha = 10^{-3}$
- $\alpha = 10^{-4}$

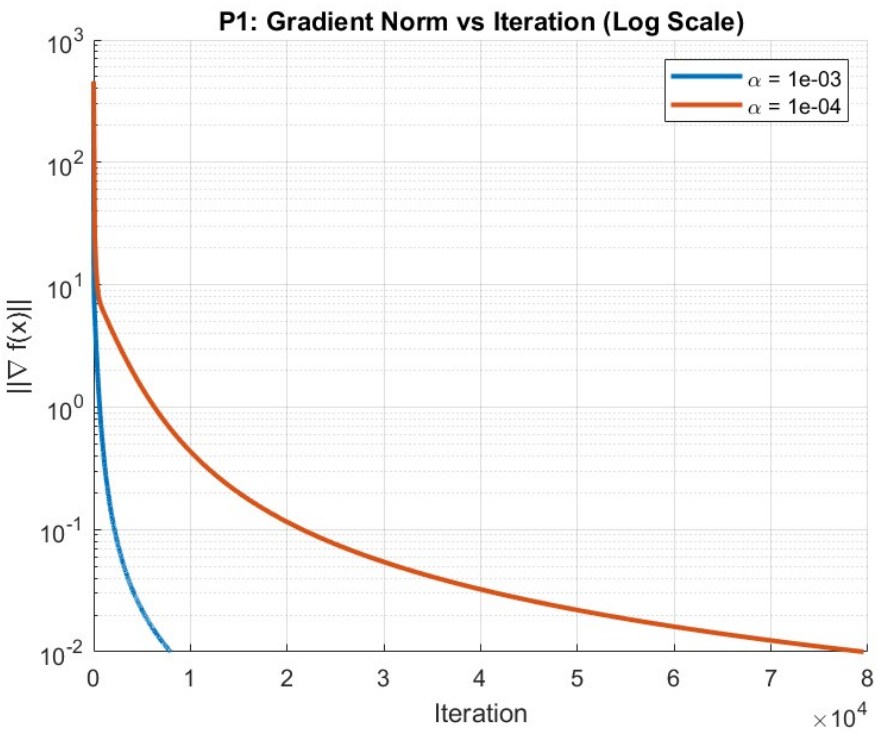
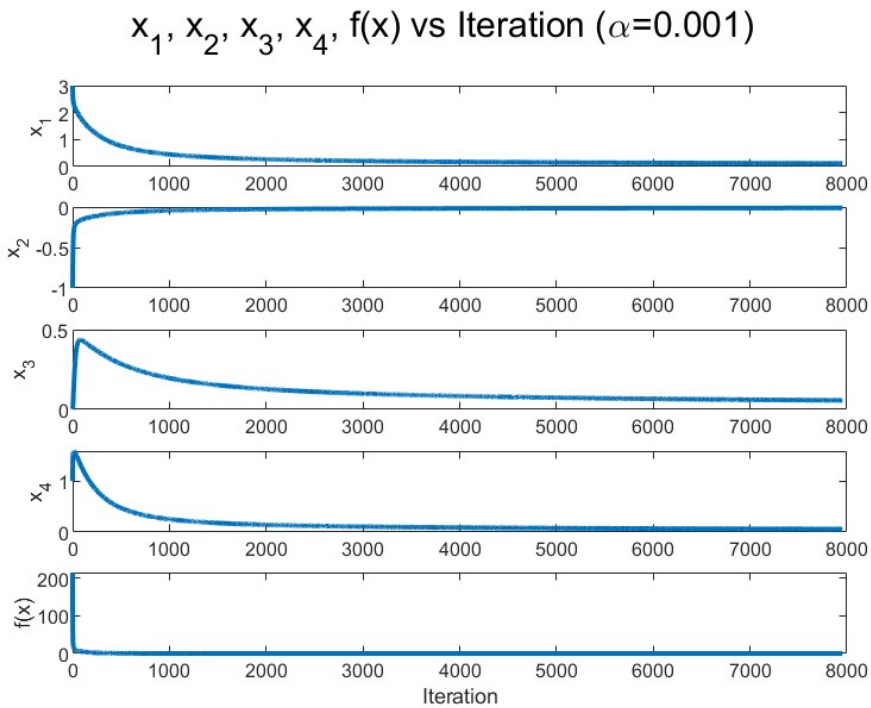
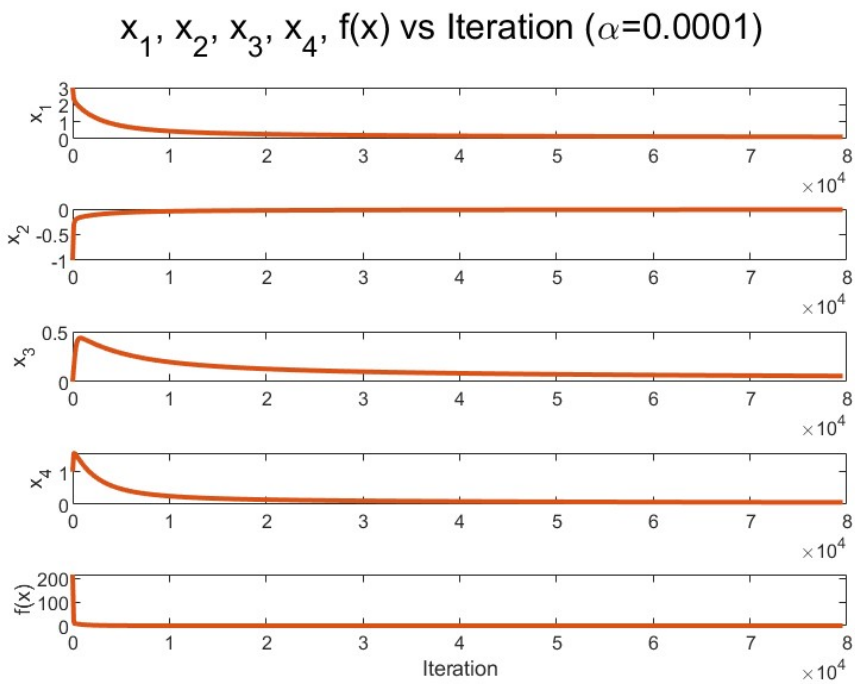


Figure 1: Gradient norm vs. iterations for gradient descent with fixed step sizes



(a) Component values and function value vs. iterations for $\alpha = 10^{-3}$



(b) Component values and function value vs. iterations for $\alpha = 10^{-4}$

Figure 3: Evolution of variables and function value for gradient descent with fixed step sizes

1.1.1 Implementation and Results

With $\alpha = 10^{-3}$, the algorithm converged in **7955** iterations to a function value of **3.561036e-04**. Using the smaller step size such as $\alpha = 10^{-4}$ required 79569 iterations to reach a function value of **3.561581e-04**

We can clearly see the impact of step size on the gradient descent algorithm. Using a smaller step size causes slower convergence we can say that with the second step size, the function moves more carefully. However, at the end of the iterations, the results of the function are very similar, even though there is more than a 10 fold difference in the number of iterations. Therefore, in this question, a smaller step size does not necessarily provide a better optimization outcome.

These results shows the importance of selecting an appropriate step size when using fixed-step gradient descent. A step size that is too small causes excessive computation (as we see at this problem), while one that is too large can lead to divergence.

1.2 Problem 2: Steepest Descent with Golden Section Line Search

The steepest descent algorithm with exact line search determines the optimal step size at each iteration by solving:

$$\alpha_k = \arg\min_{\alpha>0} f(x^{(k)} - \alpha \nabla f(x^{(k)})) \tag{3}$$

The golden section search is a technique for finding the minimum point of a unimodal function. With using the golden ratio to determine the most efficient points at which to evaluate the function.

1.2.1 Implementation and Results

Using these parameters the golden section method was implemented:

- Search interval: $[0, 0.1]$
- $\epsilon = 10^{-5}$
- $\tau = \frac{\sqrt{5}-1}{2} \approx 0.618$

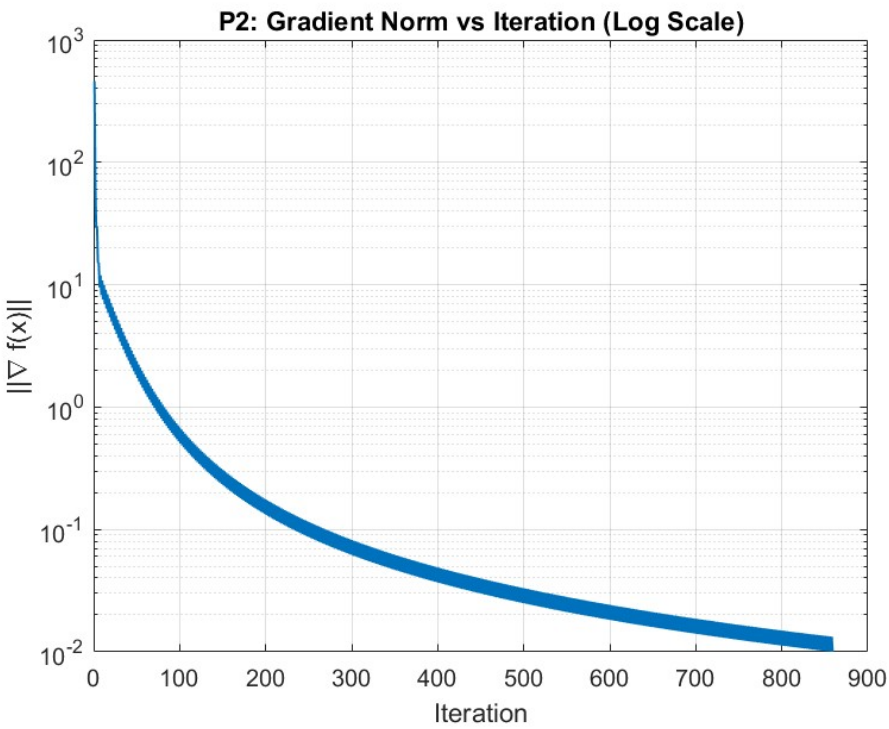


Figure 4: Gradient norm vs. iterations for steepest descent with golden section line search

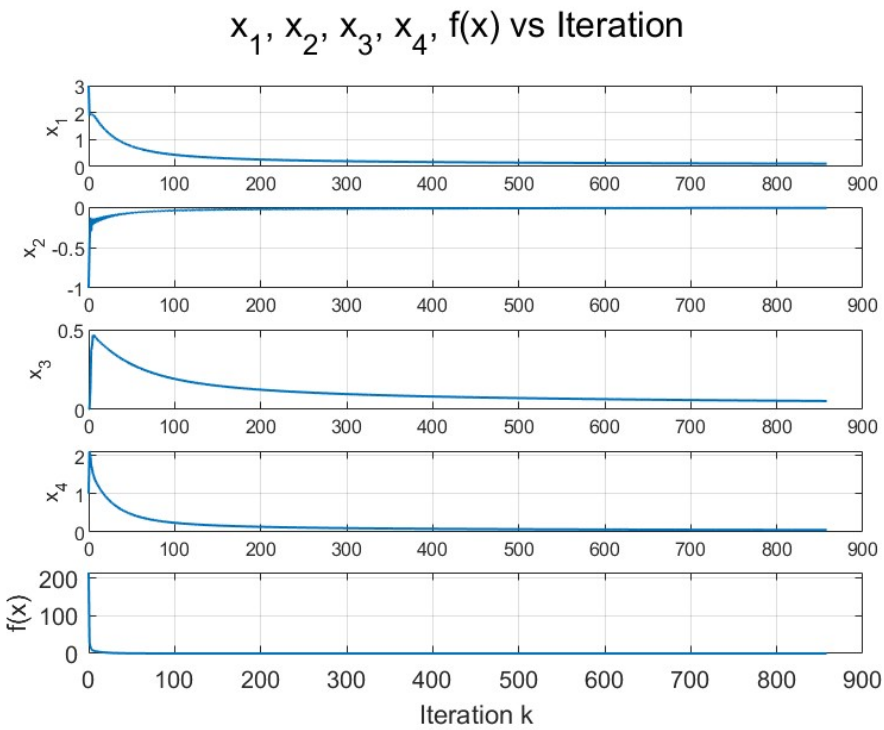


Figure 5: Component values and function value vs. iterations for steepest descent

The termination criteria reached at **859th (number of k = 859)** iteration. The total number of line search is **17160** and with **20** iterations we reached at the final step. Also the eventual value of function is **2.622807e-04**

The dramatic improvement from 7955 iterations (fixed step size) to 859 iterations with golden section line search shows the power of adaptive step sizes. This huge reduction is mainly due to golden section search since every step, we can choose dynamically optimal step size. However sum of line search is highly large number. That means, the method needs to try many values to find best step size. In short, even though the number of iteration is fewer, each iteration is more expensive computational.

1.3 Problem 3: Gradient Descent with Armijo-Goldstein Line Search

The Armijo-Goldstein conditions provide a way to find an acceptable step size without performing a full exact line search. These conditions conditions are:

$$f(x^{(k)} + \alpha_k d^{(k)}) \leq f(x^{(k)}) + \epsilon \alpha_k \nabla f(x^{(k)})^T d^{(k)} \tag{4}$$

$$f(x^{(k)} + \alpha_k d^{(k)}) \geq f(x^{(k)}) + \eta \alpha_k \nabla f(x^{(k)})^T d^{(k)} \tag{5}$$

where $0 < \epsilon < \eta < 1$.

The parameters used were:

- $\epsilon = 0.2$ (lower bound parameter)
- $\eta = 0.8$ (upper bound parameter)

- $\tau_1 = 0.5$ (reduction factor)
- $\tau_2 = 1.5$ (expansion factor)
- Initial step size $\alpha_{init} = 1$

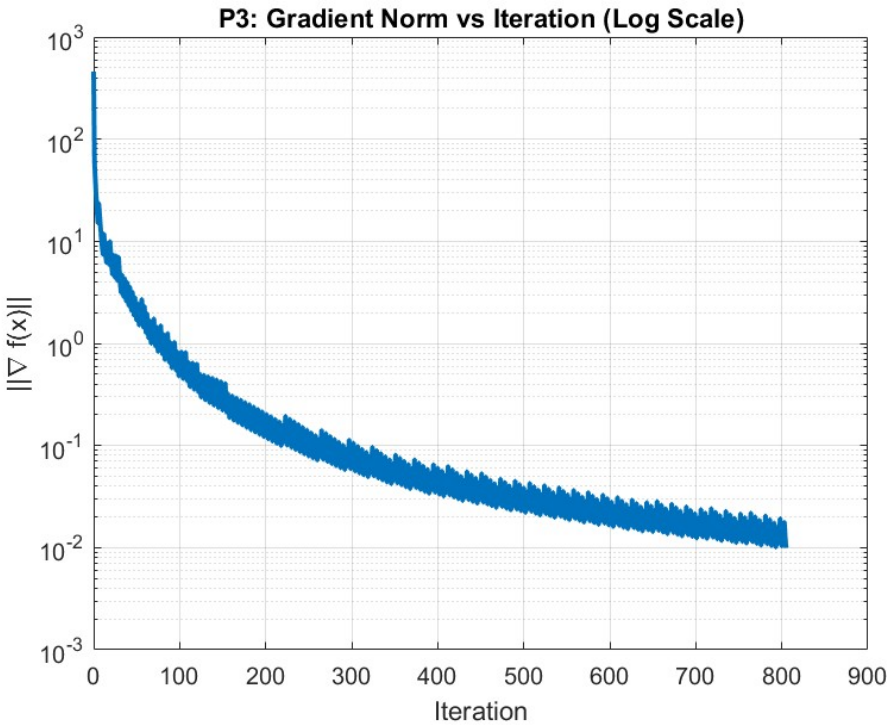


Figure 6: Gradient norm vs. iterations for gradient descent with Armijo-Goldstein line search

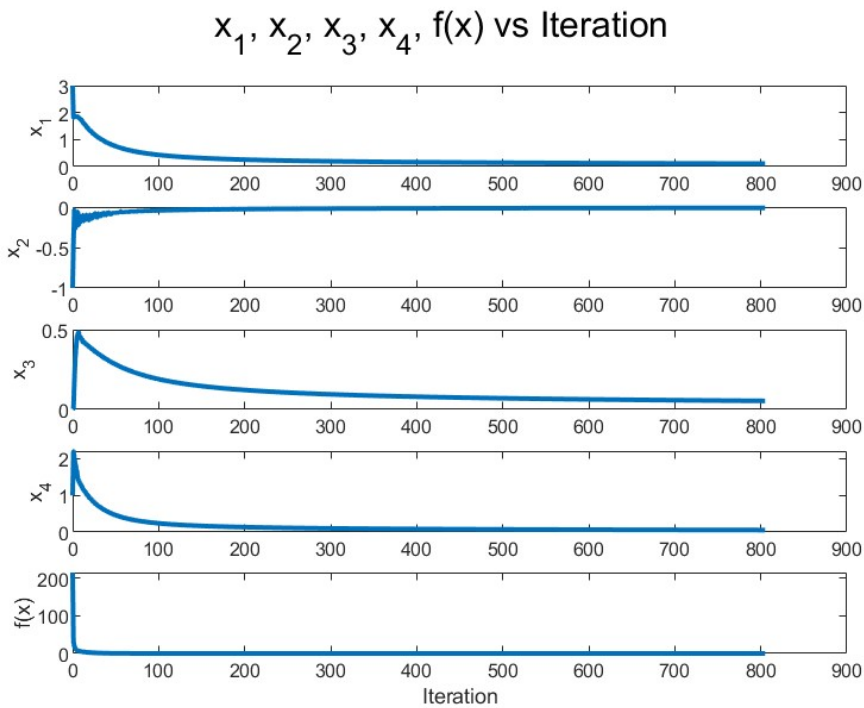


Figure 7: Component values and function value vs. iterations

1.3.1 Implementation and Results

The algorithm converged in **806** (number of k) iterations with a final function value of **2.824890e-04**. The total number of line search iterations was **5317**, with **7** iterations in the final line search step.

The Armijo-Goldstein method provides a good balance between computation cost and convergence speed. This method is so better than the method at P1 and slightly better than P2 according to iterations. When we consider the the total number of line search the Armijo-Goldstein method is more efficient. It finds a good enough step size more quickly, without needing to do an exact search like in the golden section method.

1.4 Problem 4: Newton's Method

Newton's method uses the Hessian matrix to find better descent directions:

$$x^{(k+1)} = x^{(k)} - [H(x^{(k)})]^{-1} \nabla f(x^{(k)}) \tag{6}$$

where $H(x^{(k)})$ is the Hessian matrix of f at $x^{(k)}$.

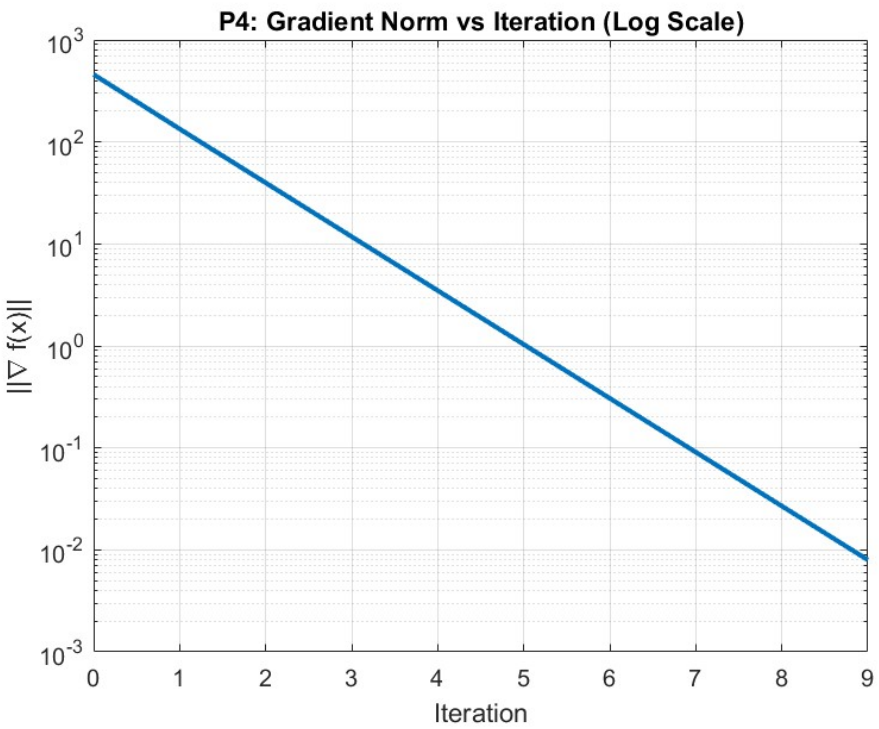


Figure 8: Gradient norm vs. iterations for Newton's method

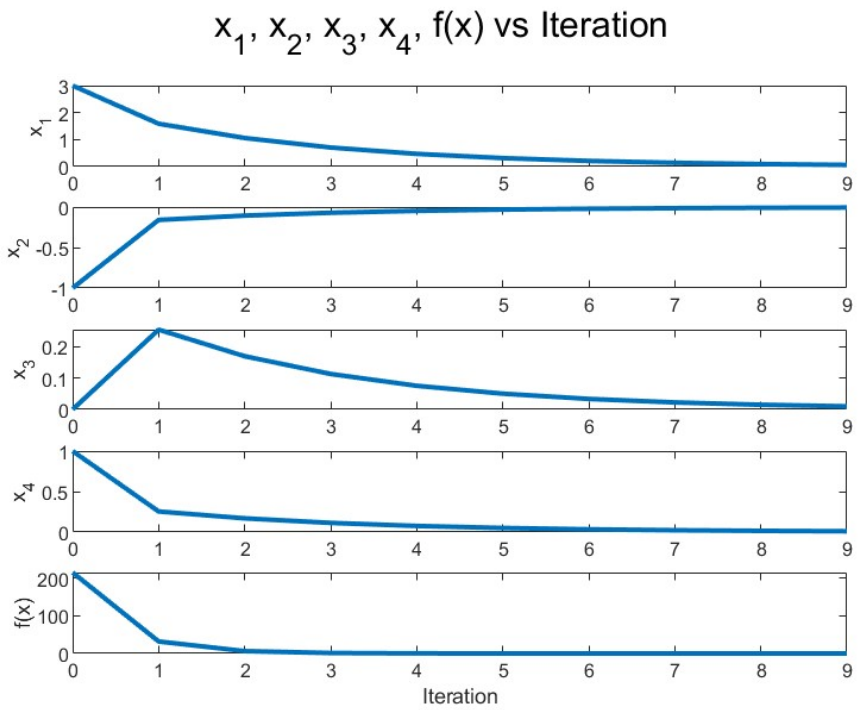


Figure 9: Component values and function value vs. iterations for Newton's method

1.4.1 Implementation and Results

Newton's method converged in only **10** iterations to a function value of **7.371240e-05**. Newton's method reached the stopping condition in just 10 iterations, which is much faster than the previous methods. This is because Newton's method uses second derivative of f function, which allows it to take more accurate steps directly toward the minimum. Wee see that Newton's method is very effective at minimizing the function.

Since Newton's method does not use a line search, there are no values for total or last line search iterations. The step size is always determined by solving the Newton system directly.

This should not be forgotten that Newton's method doesn't guarantee descent if the Hessian is not positive definite. Fortunately, for the Powell function from our starting point.

1.5 Problem 5: Modified Newton's Method with Golden Section Line Search

The modified Newton's method combines Newton's direction with a line search to determine the step size:

$$x^{(k+1)} = x^{(k)} - \alpha_k [H_f(x^{(k)})]^{-1} \nabla f(x^{(k)}) \tag{7}$$

The algorithm was implemented using the golden section line search method with a search interval of [0, 1].

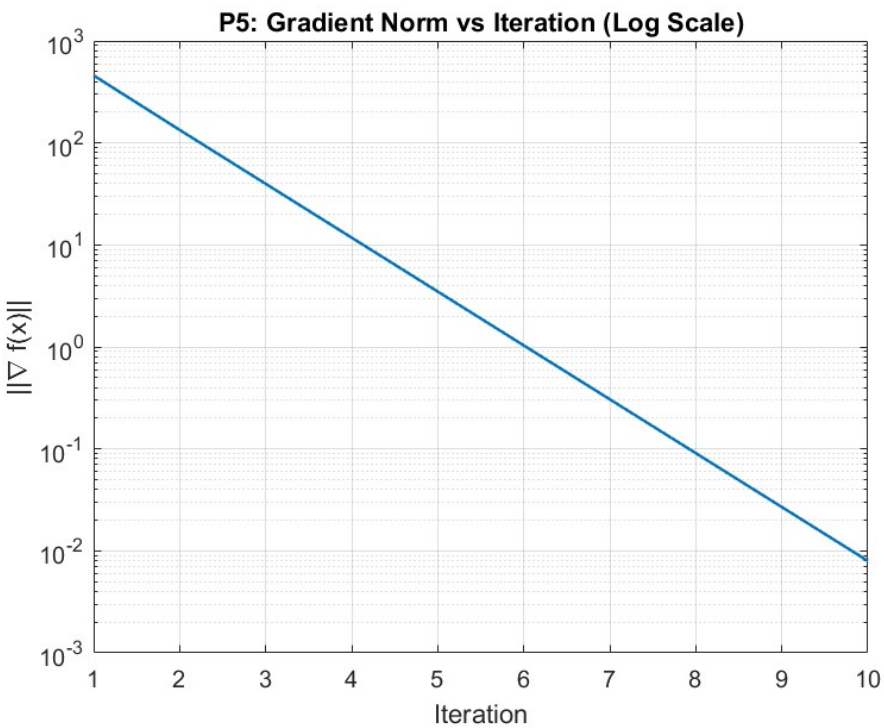


Figure 10: Gradient norm vs. iterations for modified Newton's method with golden section line search

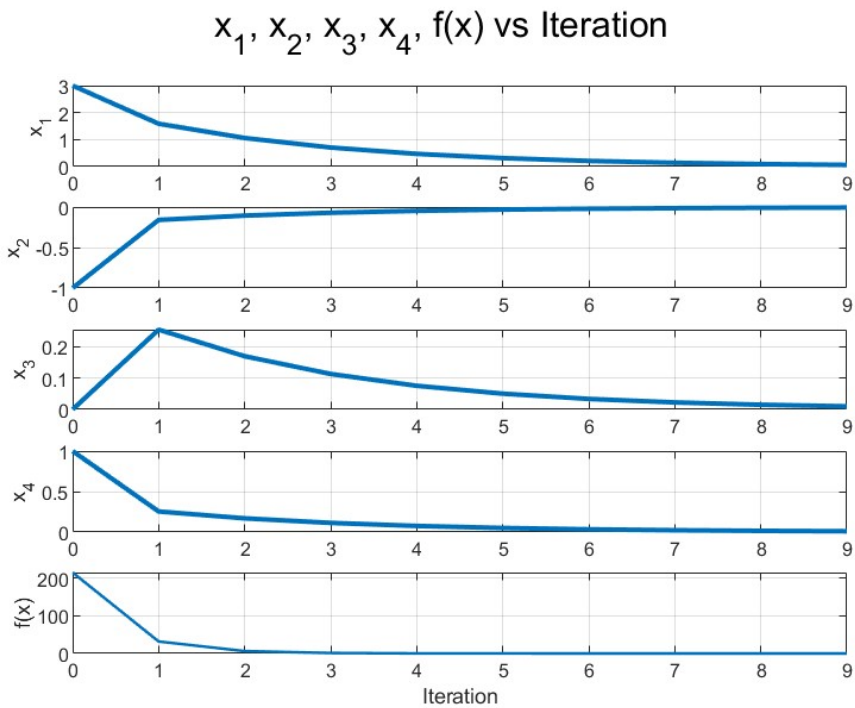


Figure 11: Component values and function value vs. iterations

1.5.1 Implementation and Results

This algorithm converged in **10** iterations just like standard Newton's method and a final function value of **7.371880e-05**. The total number of line search iterations was **216**, with **24** iterations in the final line search step.

Unlike standard Newton's method, this version uses the golden section method for exact line search. This shows that while the algorithm benefits from Newton-like directions, it still needs extra work to find the best step size at each iteration.

Compared to algorithm at P2 (steepest descent + golden section), modified Newton's method used much fewer line search steps while achieving a better function value in fewer main iterations. This makes the modified Newton method with golden section both accurate and relatively efficient.

1.6 Problem 6: Modified Newton's Method with Armijo-Goldstein Line Search

This algorithm uses Newton's direction with the Armijo-Goldstein line search conditions, combining the advantages of both approaches:

$$x^{(k+1)} = x^{(k)} - \alpha_k [H_f(x^{(k)})]^{-1} \nabla f(x^{(k)}) \tag{8}$$

The Armijo-Goldstein conditions are applied to determine a suitable step size α_k at each iteration.

The Armijo-Goldstein parameters were:

- $\epsilon = 10^{-4}$
- $\eta = 1 - \epsilon = 0.9999$
- $\tau_1 = 0.5$
- $\tau_2 = 1.5$
- Initial step size $\alpha_{init} = 1$

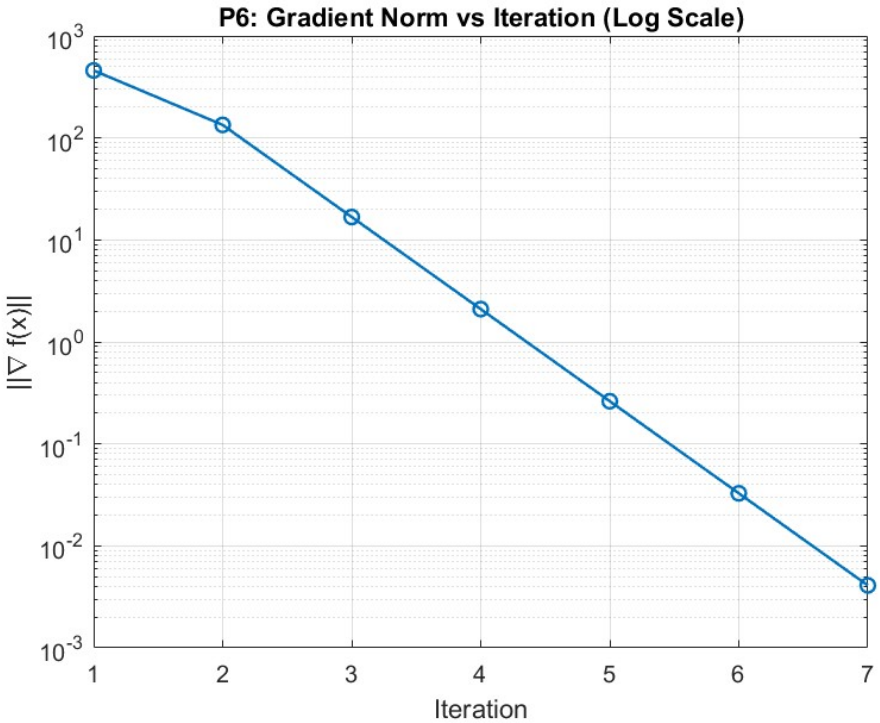


Figure 12: Gradient norm vs. iterations for modified Newton's method with Armijo-Goldstein line search

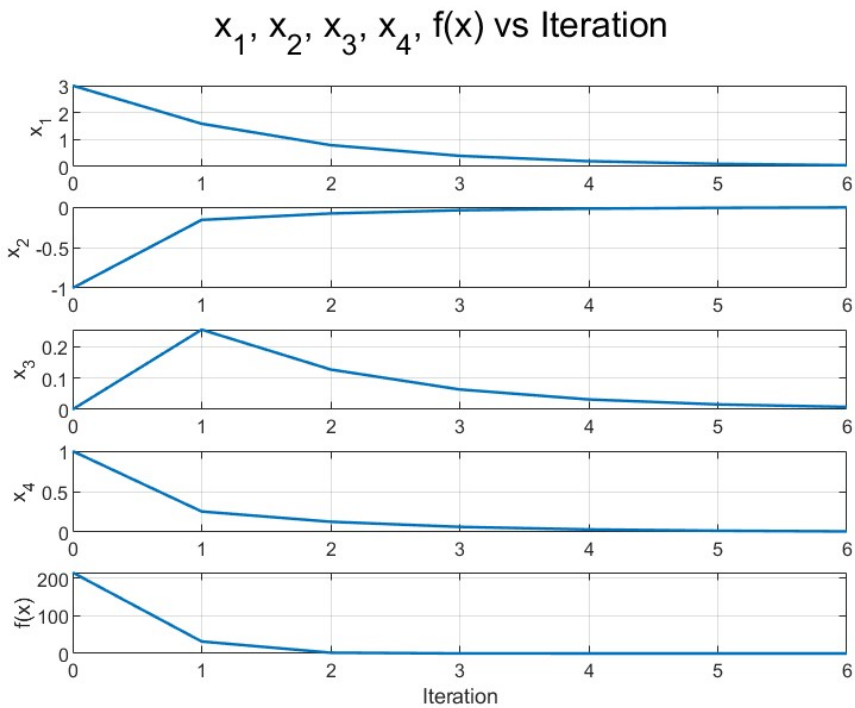


Figure 13: Component values and function value vs. iterations

This method reached the result the fastest, needing only **7** iterations to get a function value of **3.032920e-05**. It used **5** line search steps in total, with **1** in the last step.

Among all methods, the modified Newton method with Armijo-Goldstein line search turned out to be the most efficient. It completed the task in the fewest steps and with fewer line search tries. This shows that when using Newton's method, we can choose steps more carefully.

It's also important to note that the parameters used here allow only very small step ranges. While this fits well with Newton's method, which already gives good directions, it might be too limiting for other methods like gradient descent.

2 Comparison and Discussion

Table summarizes the performance of all implemented algorithms.

width=

Problem	Iteration (k)	LS Iter (Sum)	LS Iter (Last)	Eventual Func. Valu
Gradient Descent ($\alpha = 10^{-3}$)	7955	N/A	N/A	3.561036e-0
Gradient Descent ($\alpha = 10^{-4}$)	79569	N/A	N/A	3.561581e-0
Steepest Descent (Golden Section)	859	17160	20	2.622807e-0
Gradient Descent (Armijo-Goldstein)	806	5317	7	2.824890e-0
Newton's Method	10	N/A	N/A	7.371240e-0
Modified Newton (Golden Section)	10	216	24	7.371880e-0
Modified Newton (Armijo-Goldstein)	7	5	1	3.032920e-0

Table 1: Optimization results including iterations and function values

2.0.1 Convergence Speed

The number of iterations required for convergence varied significantly across methods:

- Gradient descent with fixed step sizes required by far the most iterations, especially with the smaller step size.
- Gradient descent with Armijo-Goldstein converged a little bir better Steepest descent in terms of number of iteration, and so much better in terms of total number of line search .
- Second-order methods (Newton and modified Newton) converged the fastest, requiring between 7-10 iterations.

2.0.2 Line Search Efficiency

Among the line search methods:

- Armijo-Goldstein was more efficient since it adjusts the step size based on the slope, avoiding unnecessary evaluations.
- It worked especially well with Newton's method, needing fewer total iterations to converge.
- The golden section method used more iterations because it searches over a fixed interval, which takes longer.

Function value: All methods reached similar values, showing they found the same minimum.

Sensitivity to Algorithm Choice: Some methods converge faster, so choosing the right one saves time.

Value of Second Order Derivative: Newton's method uses curvature info, so it takes smarter steps and converges quickly.

Parameter sensitivity: If the step size or line search settings are bad, the method becomes slow or unstable.

2.1 Conclusions

Based on the experimental results, we can draw the following conclusions:

1. Newton's methods are significantly more efficient in terms of the number of iterations required for convergence, demonstrating the value of second-order information in optimization.
2. Modified Newton's method with Armijo-Goldstein line search achieved the best overall performance, requiring both the fewest iterations and relatively few line search evaluations.
3. For gradient-based methods, line search dramatically improves convergence speed compared to fixed step sizes.
4. The Armijo-Goldstein line search provides a good balance between accuracy and efficiency compared to the exact golden section method.
5. All methods successfully converged to approximately the same solution, validating the robustness of these optimization techniques for the Powell function.

For practical applications, the choice of optimization algorithm should consider both the convergence rate and the computational cost per iteration. Based on our results, the modified Newton's method with Armijo-Goldstein line search offers the best balance for the function optimization problem.

Contributors and Task Distribution

Prepared by: Şeyma Ünal, Berk Aydemir

Task Distribution:

- **Şeyma Ünal:** Solved Questions 2, 5, and 6; reviewed Questions 1, 3, and 4; contributed to report writing.
- **Berk Aydemir:** Solved Questions 1, 3, and 4; reviewed Questions 2, 5, and 6; merged the final code.